

Programmeringsparadigmer

Statisk og dynamisk typning

Hans Hüttel

15. september 2025

I denne korte note vil jeg diskutere fordele og ulemper ved statisk og dynamisk typning. Den kano-niske reference til dette meget omfattende emne er den fremragende bog af Pierce om typesystemer i programmeringssprog [7].

1 Hvad er typesystemer?

De fleste programmeringssprog i dag har et typesystem, der definerer en måde at klassificere syntaktiske enheder på. Nogle sprog som C har meget simple typesystemer, mens andre, især funktionelle sprog som sprogene i ML-familien og Haskell, har meget udtryksfulde typesystemer. Sprogene i Lisp-familien har også et typebegreb typer, men det er meget anderledes – Lisp-sprogene er dynamisk typede. Nogle meget anvendte programmeringssprog har slet ingen typesystemer. Et velkendt eksempel på det er JavaScript.

Traditionelt set har typesystemer været brugt til at klassificere programmer ved hjælp af *typetjek*: givet en tildeling af typer til bestanddele i et program, undersøger typetjek, om P er *veltypet*, dvs. ikke indeholder typefejl.

Implementationer af typede programmeringssprog indeholder altid en form for typetjek. Den væsentlige skelnen er mellem om typetjek sker *før* køretid eller *på* køretid.

2 Statiske og dynamiske typesystemer

Et statisk typesystem udfører typetjek baseret på programkoden *før køretid*. Typefejl bliver så fanget som del af den semantiske analyse. C, Java, ML-sprogene, Scala og Haskell har alle statiske typesystemer.

Et dynamisk typesystem fanger typefejl *på kørselstidspunktet*. Sprog som Lisp-dialekterne, Python, Ruby og Lua er dynamisk typede. Hvis en typefejl bliver fundet på kørselstidspunktet, stopper udførelsen af programmet.

I nogle sammenhænge har der været heftige diskussioner om, hvorvidt statiske eller dynamiske type-discipliner er bedst. Et meget vigtigt argument for at have en statisk type-disciplin er, at hvis et statisk typesystem kan vises at være *sikkert*, så vil visse køretidsfejl aldrig kunne forekomme.

Et program har en køretidsfejl, hvis der på et tidspunkt under dets udførelse bliver umuligt at fortsætte udførelsen. Det sker normalt fordi programmet forsøger at udføre en operation, der ikke giver mening – for eksempel en multiplikation af en streng og en sandhedsværdi.

Lad os er antage, at programmer (som i Haskell) er udtryk, og at deres semantik er givet ved en small-step-semantik, hvor transitioner er på formen $e \rightarrow e'$. En sådan transition skal læses som at e kan tage et enkelt skridt, hvorefter det er e' , der skal evalueres. Vi skriver $e \rightarrow$, hvis e kan udføre et skridt, og $e \rightarrow^* e'$, hvis e kan evaluere til e' i nul eller flere skridt.

For ethvert programmeringssprog, der er Turing-fuldstændigt (dvs. at enhver Turing-maskine kan simuleres med et program i sproget), og enhver ikke-triviell klasse af fejl er det uafgørbart, om et givet program er frit for fejl.

Sætning 1. Antag, at vores programmeringssprog er Turing-fuldstændigt. Lad C være en klasse af fejl, og lad e være et udtryk. Det er uafgørbart, om der findes et e' , således at $e \rightarrow^* e'$, hvor e' udfører køretidsfejlen C .

Bevis: En reduktion fra standseproblemet for Turingmaskiner. Vi antager (uden tab af generalitet), at da vores programmeringssprog er Turing-komplet, så findes der en oversættelse fra par $\langle M, w \rangle$ af Turingmaskiner og strenge til programmer $e_{M,w}$. Vi antager også, at vores sprog har en opfattelse af sekventiell sammensætning, således at vi kan udføre e_1 før e_2 . Vi skriver dette som $e_1; e_2$.

Vi ved også, at der må være et program e_C , der fører til en køretidsfejl fra klassen af køretidsfejl C .

Givet beskrivelsen $\langle M, w \rangle$ af en Turingmaskine M og en inputstreng w , kan vi konstruere et program $e_{M,w}$, der simulerer opførslen af M , når den køres med input w .

Nu vil programmet $e_{M,w}; e_C$ resultere i en køretidsfejl, hvis og kun hvis M standser, når den får input w . □

3 Hvad et statisk typesystem garanterer

Den vigtige observation er, at mens det er uafgørbart, om et program vil give anledning til en køretidsfejl, er det for alle statiske typesystemer faktisk afgørbart, om et program er veltypet. Derfor kunne man håbe, at uanset hvilket typesystem vi betragter, bør det være i stand til at give en sund karakterisation af en klasse C af køretidsfejl. Hvis det er tilfældet, ved vi, at hvis et program er veltypet, vil det ikke give anledning til en køretidsfejl i C .

Robin Milner kom med sloganet at *veltypedede programmer kan ikke køre galt* i en vigtig artikel 1978. Dette betyder, at hvis et program er veltypet, vil det aldrig "sidde fast". Hvis denne egenskab gælder for vores typesystem, siger vi at det er *typesikkerhed*

Typesikkerhed er en egenskab, som ethvert statisk typesystem bør garantere — men det er ikke altid tilfældet. Et eksempel på et typesystem, der ikke har denne gode egenskab, er typesystemet for C.

I forbindelse med statiske typer definerer vi typesikkerhed i forhold til semantikken i det programmeringssprog, vi overvejer.

Lad os antage, at typedomme i vores typesystem er på formen $E \vdash e : t$. Dette skal læses som at, givet typekonteksten E , der fortæller os typerne af de frie variable i e , så har udtrykket e typen t .

Så gælder typesikkerhed for vores typesystem i forhold til semantikken, hvis følgende to egenskaber gælder.

Typebevarelse Hvis $E \vdash e : t$ og $e \rightarrow e'$, så har vi, at $E \vdash e' : t$

Fremskridt Hvis $E \vdash e : t$, så er e enten en værdi, eller $e \rightarrow$.

Typebevarelse fortæller os at et veltypet udtryk vil forblive veltypet efter et beregningsskridt.

Fremskridtegenskaben garanterer, at et veltypet udtryk ikke kan "sidde fast": Enten er udtrykket fuldt evalueret, eller også kan evalueringen fortsætte.

Tilsammen medfører disse to egenskaber, at et veltypet udtryk aldrig vil kunne "sidde fast".

4 Typetjek og typeinferens

Når man bruger statisk typning, finder typetjek sted som en del af den semantiske analyse i front-end. Af denne grund er et naturligt krav til et typesystem, at der skal eksistere en algoritme, der kan afgøre, om et program P er veltypet.

I kølvandet på Milners arbejde fra 1978 er begrebet *typeinferens* blevet stadig vigtigere i denne sammenhæng. Typeinferens kan vi se som det omvendte problem til typetjek: Givet et program P , kan vi konstruere en typekontekst, der vil gøre P veltypet? Implementationerne af af mange funktionelle sprog, herunder ML-familien og Haskell, har typeinferens som et centralt aspekt.

Hvis typesystemet for et programmeringssprog understøtter typeinferens, kan vi tillade *implicit typning*, dvs. at vi kan lade være med at programmet med typeinformation, hvis ikke vi vil. Hvis vi dermod kræver at typeannoteringer skal være der, taler vi om *eksplicit typning*. Haskell er et eksempel på et sprog, der tillader begge tilgange — takket være typeinferens.

Vi ønsker, at typetjek skal være afgørbart, dvs. vi ønsker, at der findes en algoritme, der, når den får en E , et udtryk e og en type t , kan fortælle os, om $E \vdash e : t$.

Hvis typetjek er afgørbart, og vores statiske typesystem er sikkert, har vi situationen som vi kan se i figur 1. Vi er i stand til at afgøre, om et program er veltypet, men fordi det er uafgørbart, om et program vil "sidde fast" på køretid mht. den egenskab, som vores typesystem skal forhindre, må der være programmer, der er fejlfri, men ikke veltypedede. Vi kalder denne "gråzone" for typesystemets *slack*. Der er altid uendeligt mange programmer i slack (hvorfor egentlig?).

På den anden side kan et sikkert statisk typesystem ændre programudviklingsprocessen: Vi skal bruge mindre tid på fejlfinding og testning, fordi vi ved, at visse køretidsfejl ikke kan optræde, hvis vores program er veltypet. Til gengæld skal vi konstruere et program, der er veltypet, og det betyder at arbejdsindsatsen for programmøren flyttes fra test-fasen over til selve det at fjerne typefejl inden programmet kan køre.

Example 1. I Haskell er et eksempel på et udtryk, der findes i slack i typesystemet:

```
if 2 + 2 == 4 then "mango" else (not 484000)
```

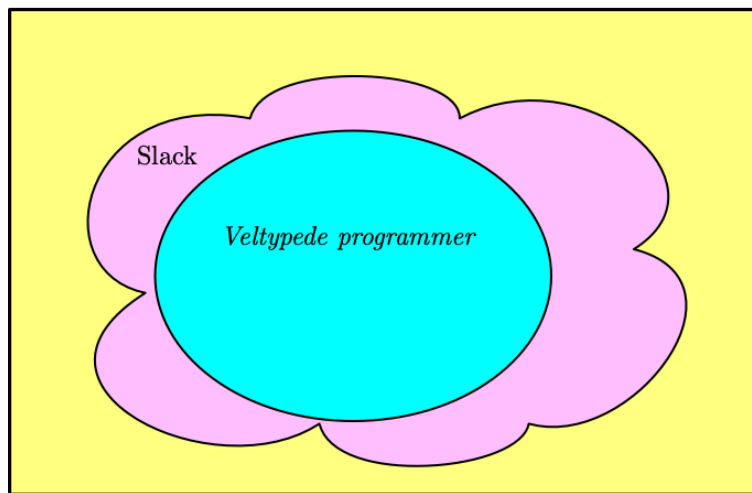
De to grene af det betingede udtryk har forskellige typer, men den første gren vælges altid, da betingelsen altid er sand — ingen køretidsfejl ville forekomme, hvis Haskell tillod dette at evaluere.

5 Fordele og ulemper

Jeg har allerede skitseret den vigtige fordel, som sikre statiske typesystemer vil have. Men nogle mennesker argumenterer for, at der er fordele ved *ikke* at have et typesystem — i den forstand, at man slet ikke behøver at tænke på typer. Og atter andre argumenterer for at dynamisk typning er bedst.

Her er nogle hovedargumenter for henholdsvis statisk og dynamisk typning/ingen typning.

Alle programmer



Figur 1: Slack i et typesystem

5.1 Argumenter for statisk typning

Udtryksfuldhed Programmer, der ikke er veltype og afvises af typetjek, er som regel meningsløse

Tidlig advarsel En statisk analyse kan forhindre os i at køre programmer, der indeholder alvorlige fejl, og hjælpe os med at finde fejlene, mens tid er.

Typer som specifikationer Typer beskriver programmets adfærd, og vi kan derfor bruge dem som specifikationer, når vi udvikler et program. Nogle typesystemer er meget udtryksfulde og kan fange en stor klasse af køretidsfejl.

Læsarhed Et program med typeinformation kan være mere læsbart, fordi vi gør klassificeringen af værdier eksplicit.

Homogene datastrukturer Datastrukturer skal være homogene, dvs. alle elementer i en datastruktur skal have samme type. Dette sikrer, at operationer på datastrukturer ikke kan gå galt.

Effektivitet Hvis man bruger et statisk typesystem, skal typetjek også kun ske én gang, nemlig på oversættelsestidspunktet, hvorimod en dynamisk type-disciplin kræver, at køretidssystemet foretager typetjek hver gang et program bliver kørt.

5.2 Argumenter for dynamisk typning eller ingen typning

Sen advarsel er bedre Fortalere for dynamisk typning siger ofte, at de køretidsfejl, der virkelig betyder noget, slet ikke er typefejl, så det giver mere mening at håndtere dem ved kørselstidspunktet. Ifølge dem er det mere nyttigt at have en god programmeringsomgivelse til fejlfinding og testning.

Produktivitet Hvis man ikke behøver at bekymre sig om typefejl ved kompileringstidspunktet, vil man bruge mindre tid på at udvikle et program, der kan udføres.

Evolution Hvis man bruger dynamisk typning, er det nemt at ændre typen af et x . Men i et statisk typet sprog kan ændring af typen af x kræve, at programmøren opdaterer typeannotationer for mange andre steder i programmet, der afhænger af x .

Heterogene datastrukturer Det bliver muligt at have heterogene datastrukturer, dvs. datastrukturer, hvor ikke alle elementer har samme type. Dette gør det muligt at skrive enklere kode til behandling af datastrukturer.

Læsarhed Fortalere for dynamisk typning siger ofte, at typeinformation er besværlig og faktisk formindsker læsarhed af et program. Programkoden bliver længere, fordi vi skal annotere den med typer.

5.3 Dynamisk typning er en kedelig form for statisk typning (??)

Robert Harper, som er en af designerne af Standard ML, er kendt for at sige, at et dynamisk typet sprog er et statisk typet sprog med kun én statisk type!

Han skriver [3]

The characteristics of a dynamic language are the same, values are classified by into a variety of forms that can be distinguished at run-time. A collection of values can be of a variety of classes, and we can sort out at run-time which is which and how to handle each form of value. Since every value in a dynamic language is classified in this manner, what we are doing is agglomerating all of the values of the language into a single, gigantic (perhaps even extensible) type. To borrow an apt description from Dana Scott, the so-called untyped (that is “dynamically typed”) languages are, in fact, untyped. Rather than have a variety of types from which to choose, there is but one!

Antag, at vi havde et dynamisk typet sprog, hvor de eneste værdier var sandhedsværdier og heltal. Vi kunne så definere en type `ValueType` som

```
data ValueType = TV Bool | IV Integer
```

Hvis et udtryk i vores dynamisk typede sprog ikke giver en køretidsfejl, men evaluerer til en værdi, vil det altid have typen `ValueType`! På en måde er dette den samme observation som ideen om at have en særlig type kaldet `Dynamic`, som vi nævner i det følgende afsnit.

6 Kombination af det statiske og det dynamiske

Der er tilgange, der ligger et sted mellem traditionel statisk typning og dynamisk typning. En af dem er ”blød typning”, hvor ideen er at bruge et statisk typesystem til at give råd, som programmøren derefter kan vælge at se bort fra.

I sådan et system skriver programmøren sin kode, som om hun brugte et dynamisk typet sprog, og bruger derefter typeinferens til at forsøge at inferere typer for programmet og opdage typefejl. Hvis og når en sådan fejl findes, kan programmøren vælge at rette den eller lade den være og køre programmet alligevel. Så her er typefejl ikke fejl, men bare advarsler.

Ikke alle synes godt om den bløde tilgang. Krishnamurti skrev:

Every few years, yet another scripting language’s community seems to discover soft typing. Maybe because I’m the PLT loudmouth, they inevitably end up in a long thread with me. I’ve been through this with Guile, Python and Tcl. Every time, some smart, bright-eyed and bushy-tailed person sets out to build a soft typer for their language. Every time, a few weeks in, they finally realize why this is such a hideously difficult undertaking, and give up.

En mere succesfuld tilgang er *gradvis typning*, som skyldes Siek og Taha [9]. Her indfører man en ”ukendt”type ?, der kan bruges til at annotere bestanddele i et program, hvis type endnu ikke er kendt. Denne form for typesystem tillader programmøren at introducere typer i koden efterhånden som hun finder ud af mere om den tilsigtede opførsel af det program, hun skriver. Hvis alt er typet med kendte typer, vil programmet have de samme sikkerhedsgarantier som et normalt statisk typesystem ville. TypeScript er en gradvist typet version af JavaScript.

7 Dynamisk typning i Haskell?

Haskell bruger statisk typning, men der er udvidelser af Haskell, der tillader dynamisk typning. Ideen med at have en særlig type kaldet `Dynamic` i et ellers statisk typet sprog går tilbage til arbejdet af Abadi et al. [1].

`Dynamic`-grænsefladen understøtter til dynamiske typer. Ved hjælp af dette bibliotek kan man injicere værdier af vilkårlig type i en dynamisk typet værdi, `Dynamic`, og der er operationer til at konvertere dynamiske værdier til en konkret (ikke-polymorf) type.

Peterson beskriver en tilgang til dynamisk typning i Haskell [6], og Shields et al. [8] beskriver en enkelt typeinferensalgoritme, der kan udføres delvist ved kompileringstidspunktet og delvist på køretid. Dette tillader Haskell at bruge typer, der kun er kendt på køretid.

Litteratur

- [1] Martín Abadi, Luca Cardelli, Benjamin Pierce og Gordon Plotkin. 1991. Dynamic typing in a statically typed language. *ACM Trans. Program. Lang. Syst.* 13, 2 (April 1991), 237–268. <https://doi.org/10.1145/103135.103138>
- [2] `Data.Dynamic`. base-4.19.0.0: Basic libraries. <https://hackage.haskell.org/package/base-4.19.0.0/docs/Data-Dynamic.html>
- [3] Robert Harper. Dynamic Languages are Static Languages. Marts 2011. <https://existentialtype.wordpress.com/2011/03/19/dynamic-languages-are-static-languages/>

- [4] Shriram Krishnamurti. RE: bytecode unification for scripting (and other!) languages. The PLT Scheme mailing list. 15. august 2001, <https://www-old.cs.utah.edu/plt/mailarch/plt-scheme-2001/msg00788.html>
- [5] Robin Milner. A theory of type polymorphism in programming, *Journal of Computer and System Sciences*, Volume 17, Issue 3, 1978, Pages 348-375, ISSN 0022-0000, [https://doi.org/10.1016/0022-0000\(78\)90014-4](https://doi.org/10.1016/0022-0000(78)90014-4).
- [6] John Peterson. Dynamic Typing in Haskell. Research Report YALEU/DCS/RR-1022. Yale University, 2003. <http://www.cs.fsu.edu/~langley/COP4020/2019-Fall/Lectures/tr1022.pdf>
- [7] Benjamin Pierce. Types and Programming Languages, MIT Press 2000.
- [8] Mark Shields, Tim Sheard, og Simon Peyton Jones. 1998. Dynamic typing as staged type inference. In *Proceedings of the 25th ACM SIGPLAN-SIGACT symposium on Principles of programming languages (POPL '98)*. Association for Computing Machinery, New York, NY, USA, 289–302. <https://doi.org/10.1145/268946.268970>
- [9] Jeremy G. Siek og Walid Taha. Gradual Typing for Functional Languages. In *Proceedings of Scheme and Functional Programming Workshop*. ACM, pp. 81–92, 2006.